Graphics sec

sec 1

* Computer graphics: means creation, storage and maipulation of models & images

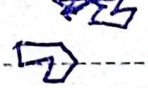
* computergraphic models → geomtric: describe how object appears (Color, Texture, Slope)
  → mathimatical: describe how object moves

* pixel: is the smallest element in Raster image

* frame buffer: Contain all pixels intensity values
  no of bits/pixel = depth of buffer = no of bit planes
  frame buffer depth: no of bits per pixel = total no of diffront colors that can be displayed.

* poly line: Connected sequance of straight lines

* polygon Classification → Simple
  → Complex
  → Concave
  → Conved



DAWENHA designs

CamScanner

* Frame buffer : Contain all pixel intensity values

See 2 * no of bits/pixel = depth of buffer = no of bits/plane

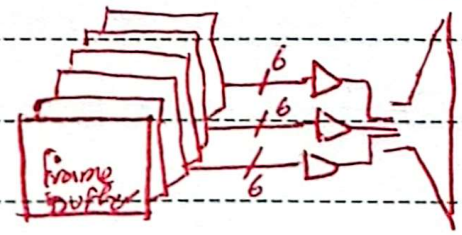
* pixel Color Representation (Color Palette)

* Color Palette : the set of Colors that display sys Can use to represent an image

* Two Common methods are used :

[1] Direct Coding : each pixel store its own Color value directly

* if frame buffer support 6 planes of memory (2 for each Color) then 2^2 level of R, G & B



* for true Color image, Color depth 24 bit means 24 planes of memory (8 for each Color) then 2^8 level of R, G, B, Total $2^{24} \simeq 16$ M Color pallette

Ex. if we use direct Coding RGB with 3 bits per primary Color how many possible Colors do we have for each pixel

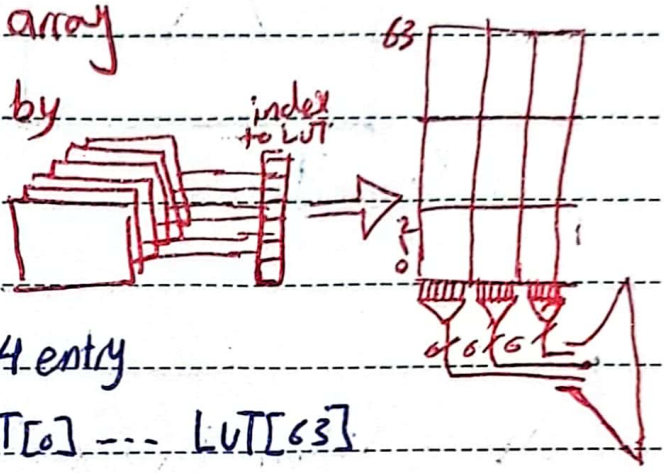
Sol

no of Colors for each pixel = $2^3 \times 2^3 \times 2^3 = 2^9 = 512$ Color

(2) indexed Color & Look up table (LUT)

instead of storing the full color value in each pixel, the pixel hold an index that points to a Color in LUT.

* reduce memory & support large no of Colors
* each elements in the array represent a Color, indexed by position within array



* if pixel depth = 6

the index of table is $= 2^6 = 64$ entry (no of entries) Say LUT[0] --- LUT[63]

EX.2 if we use 10 bits/pixel value in 24 bit LUT how many entries does LUT have & how much size of it in bytes

Sol no of entries $= 2^{\text{no of bit/pixel}} = 2^{10} = 1024$ entry

LUT size $= (\text{no of entries} * \text{no of bits in LUT}) / 8 = (2^{16} * 24)/8$

$= 3072$ byte

EX.3 if we use 2 bytes pixel value in a 24 bit LUT representation, how many entries & size of LUT.

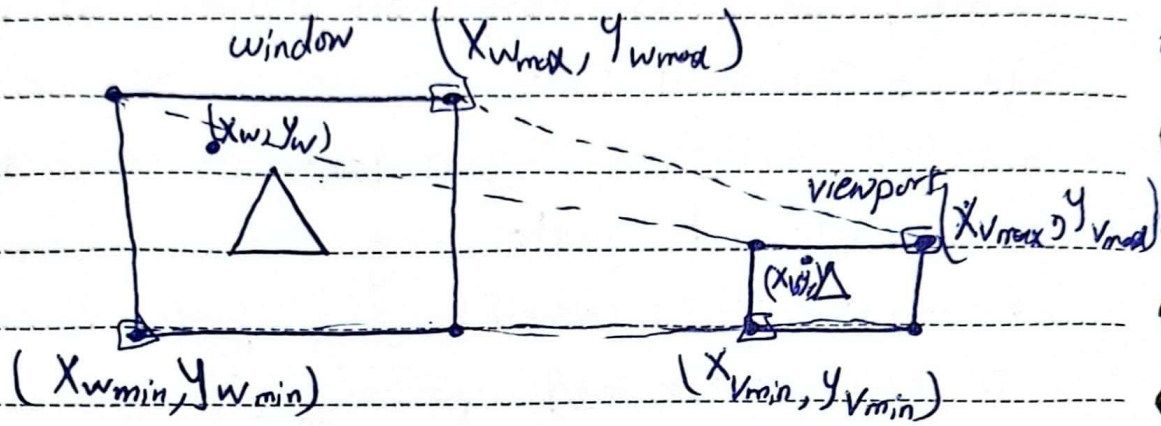
Sol 2 bytes $= 2*8 = 16$ bit

LUT entries $= 2^{16} = 65536$ entry

LUT size $= (2^{16} * 24)/8 = 196\,608$ byte

Sec 3

* window to viewport mapping



$$X_v = X_{vmin} + (S_x * (X_w - X_{wmin}))$$

$$Y_v = Y_{vmin} + (S_y * (y_w - y_{wmin}))$$

where S_x, S_y are scaling factor

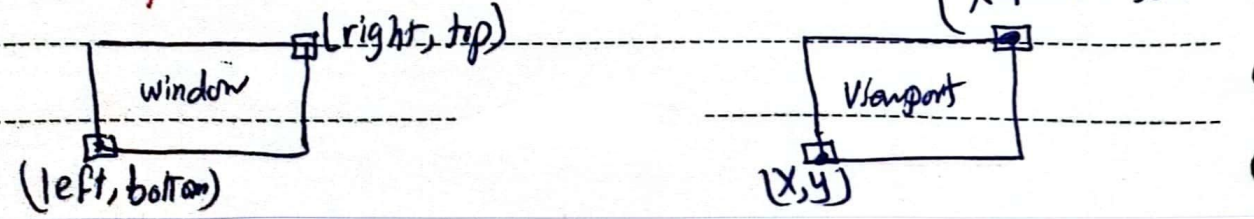
$$S_x = \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}} \qquad S_y = \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{vmin}}$$

Set window :

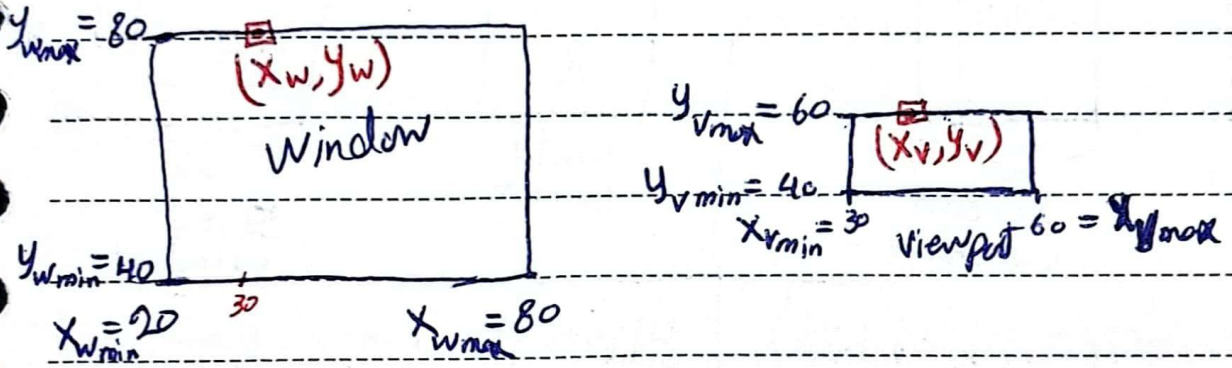
glOrtho2D(left, right, bottom, top)

Set viewport

glViewport (x, y, width, height)



Ex.1 Consider the window & viewport, the window has $(W_L, W_r, W_b, W_t) = (20, 80, 40, 80)$ & viewport $(V_L, V_r, V_b, V_t) = (30, 60, 40, 60)$ for $(X_w, y_w) = (30, 80)$ find (X_v, Y_v)



Sol

$$S_x = \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}} = \frac{60 - 30}{80 - 20} = 0.5$$

$$S_y = \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}} = \frac{60 - 40}{80 - 40} = 0.5$$

$$X_v = X_{vmin} + (S_x (X_w - X_{wmin})) = 30 + 0.5(30 - 20)$$
$$= 35$$

$$y_v = y_{vmin} + (S_y (y_w - y_{wmin})) = 40 + 0.5(80 - 40)$$
$$= 60$$

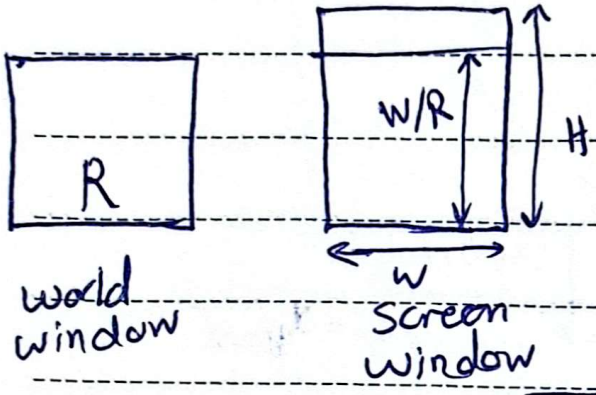
```
glOrtho2D(20, 80, 40, 80)   //Set window
glViewport(30, 40, 30, 20)  //Set viewport
```

x, y, (width) (60-30), (height) (60-40)

* setting the window & viewport Automatically:-

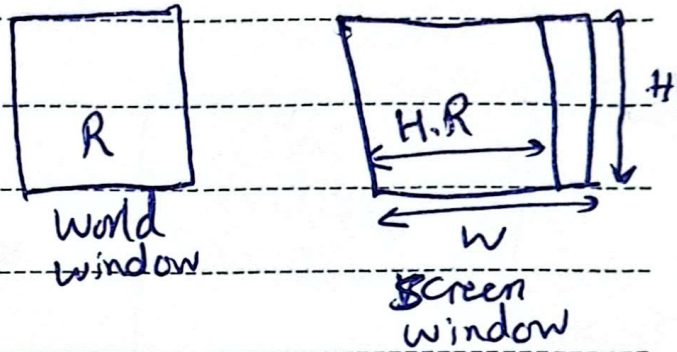
(R) aspect Ratio of world, (W, H) is width & height of screen window

if $R > W/H$



glViewport(0, 0, W, W/R)

if $R < W/H$



glViewport(0, 0, H*R, H)

Ex. 1 Find the matching viewport for a world window with aspect Ratio R = 1.6 when the Screen window has width 360 & height 200

Sol $R = 1.6 < \frac{W_v}{H_v} = \frac{360}{200} = 1.8$

then $H_v = H = 200$ & $W_v = H * R = 200 * 1.6 = 320$

glViewport(0, 0, 320, 200)

Ex.2 if $R=2$ & $W_v=360$, $H_v=200$

Sol $R=2 \Rightarrow \frac{W_v}{H_v}=\frac{360}{200}=1.8$

then $W_v=W=360$ & $H_v=W/R=\frac{360}{2}=180$

glViewport(0,0,360,180)

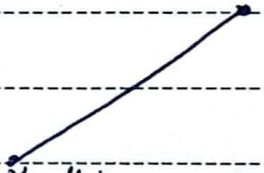
Sec 4 : line drawing

* Cases doesn't need line drawing algorithms

horizontal line	vertical line	$y = \pm x$
$m = 0$	$m = \infty$	$m = \pm 1$

* Aliasing is the jagged edges on curves & diagonal lines in a bitmap image

* Anti-aliasing : is the process of smoothing out these jaggies
  - → graphics sw prog have options for anti-aliasing text & graphics
  - → Enlarging a bitmap image accentuates (highlight) the effect of aliasing



1 DDA Algorithm

the digital differential analyzer is a scan Conversion line algorithm based on using dx or dy

Step 1 : Calculate Δx, Δy, $M = \Delta y / \Delta x$

Step 2 : Find no of steps = the greates of ($\Delta x, \Delta y$) absolute

Step 3 : لو $(\Delta x, \Delta y) > 0$ يمين ويسار ... لو $(\Delta x, \Delta y) < 0$ يسار ويمين

	Right to left	left to right
$M \le 1$	$x_{k+1} = \text{round off}(x_k - 1)$ $y_{k+1} = \text{round off}(y_k - m)$	$x_{k+1} = \text{round off}(x_k + 1)$ $y_{k+1} = \text{round off}(y_k + M)$
$M \ge 1$	$x_{k+1} = \text{roundoff}(x_k - \frac{1}{m})$ $y_{k+1} = \text{roundoff}(y_k - 1)$	$x_{k+1} = \text{roundoff}(x_k + \frac{1}{m})$ $y_{k+1} = \text{roundoff}(y_k + 1)$
$M = 1$	$x_{k+1} = \text{roundoff}(x_k - 1)$ $y_{k+1} = \text{"} \quad (y_k - 1)$	$x_{k+1} = \text{roundoff}(x_k + 1)$ $y_{k+1} = \text{"} \quad (y_k + 1)$

Start point, end point

EX.1 Calculate the points bet. $(5,6)$ and $(8,12)$ — (x_1, y_1), (x_2, y_2)

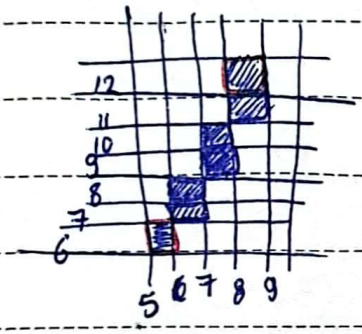
Sol ① $\Delta X = 8-5 = 3$, $\Delta Y = 12-6 = 6$, $M = \frac{\Delta Y}{\Delta X} = \frac{6}{3} = 2$

② no of steps $= \Delta y = 6$

③ $M > 1$ & drawn from left to right $= \Delta y, \Delta x > 0$

then increase $y+1$, Calculate $X = \text{round off}(X + \frac{1}{m})$ → $\frac{1}{2}$

X_k	y_k	X_{k+1}	y_{k+1}	round off (x,y)
5	6	5.5	7	(6,7)
		6	8	(6,8)
		6.5	9	(7,9)
		7	10	(7,10)
		7.5	11	(8;11)
		8	12	(8,12)



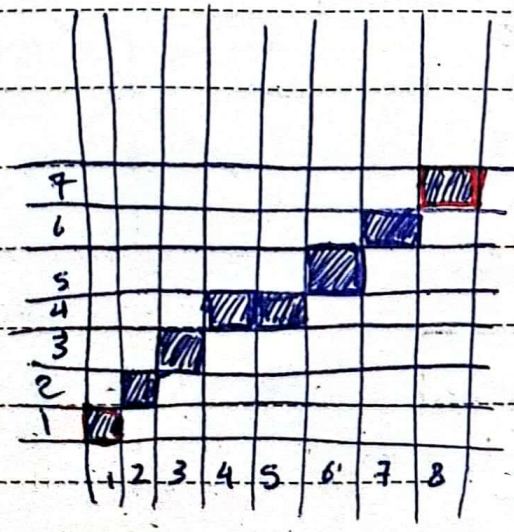
EX.2 draw line from $(1,1)$ to $(8,7)$ using DDA algorithm — (x_1, y_1), (x_2, y_2)

① $\Delta X = 8-1 = 7$, $\Delta Y = 7-1 = 6 > 0$ left to Right

② $M = \frac{6}{7} = 0.857 < 1$, no of steps $= 7$

③ increase X, Calculate $y = \text{rdff}(y+m)$

X_{k+1}	y_{k+1}	round off (x,y)
2	1.857	(2,2)
3	2.717	(3,3)
4	3.57	(4,4)
5	4.43	(5,4)
6	5.28	(6,5)
7	6.14	(7,6)
8	7	(8,7)



CamScanner

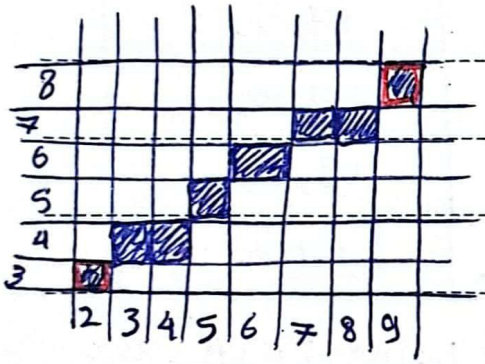
Ex 3 use DDA algorithm to draw a line from $(x_1, y_1) = (9,8)$ to $(x_2, y_2) = (2,3)$

Sol

① $\Delta X = 2-9 = -7$, $\Delta Y = 3-8 = -5 < 0$ from right to left

② $m = \frac{-5}{-7} = 0.714 < 1$ no of steps = 7

③ increas X, $y = y_k - m$



x_{k+1}	y_{k+1}	round off (x_{k+1}, y_{k+1})
8	7.28	(8,7)
7	657	(7,7)
6	5.8	(6,6)
5	5.14	(5,5)
4	4.4	(4,4)
3	3.7	(3,4)
2	3	(2,3)

2] Bresenham – line drawing algorithm.

this method is described for a line segment with $|m| = 0:1$

So we work by inc or dec. X & Calculate Y

step 1: Calculate ΔX, ΔY, P_0 (decision Parameter) $= 2\Delta Y - \Delta X$

step 2: if

①	$P_k < 0$	$P_{k+1} = P_k + 2\Delta Y$ inc(X), $y = y_{k+1}$
②	$P_k >= 0$	$P_{k+1} = P_k + 2\Delta Y - 2\Delta X$ inc(x), inc(y)

step 3: repeat until endpoint is reached or no of iterations $= (\Delta X - 1)$ times

if $|\Delta y| > |\Delta X|$ swap
if $\Delta X < 0$ dec
$\Delta y < 0$ dec

$2\Delta y = 16$ ، $2\Delta y - 2\Delta x = -4$

Ex.1 start point $(x_1, y_1) = (20, 10)$ endpoint $(x_2, y_2) = (30, 18)$

Sol $\Delta x = 30 - 20 = 10$, $\Delta y = 18 - 10 = 8$

$p_0 = 2\Delta y - \Delta x = 16 - 10 = 6 >= 0$? inc x, inc y

	P_{k+1}	X	y	setpixel (X,y)
	-	20	10	(20,10)
Case 2	6	21	11	(21, 11)
Case 2	2	22	12	(22,12)
Case 1	-2	23	12	(23, 12)
Case 2	14	24	13	(24, 13)
Case 2	10	25	14	(25 ,14)
Case 2	6	26	15	(26, 15)
Case 2	2	27	16	(27, 16)
Case 1	-2	28	16	(28 , 16)
Case 2	14	29	17	(29 ,17)
Case 2	10	30	18	(30,18)

CamScanner

Ex.2 1st point $(x_1, y_1) = (3, 32)$ & 2nd point $(x_2, y_2) = (8, 38)$

① $\Delta X = 8-3 = 5$, $\Delta y = 38-32 = 6$

$\Delta y > \Delta X$ ← (swap x, y)

(32, 3) & (38, 8) $\Delta X = 6$, $\Delta y = 5 > 0$ (in I only)

then $P_0 = 2\Delta y - \Delta X = 10-6 = 4$ Case 2 ⓟ $P_{k+1} = P_k$ → 2 ($2\Delta y - 2\Delta X$)

Case 1 ⓞ $P_{k+1} = P_k + 10$ → $2\Delta y$

	P_{k+1}	X	y	Point (y, x)
	–	32	3	(3, 32)
Case 2	4 →	33	4	(4, 33)
"	2 →	34	5	(5, 34)
"	0 →	35	6	(6, 35)
Case 1	−2 →	36	6	(6, 36)
Case 2	8 →	37	7	(7, 37)
"	6 →	38	8	(8, 38)

Ex.3 (8, 32) & (3, 38)

① $\Delta X = 3-8 = -5$, $\Delta y = 38-32 = 6$

$|\Delta y| > |\Delta X|$ ← swap (x, y)

(32, 8) & (38, 3) ← ΔX +ve, Δy = |−5| → dec y

$P_0 = 2\Delta y - \Delta X = 10-6 = 4$

P_{k+1} ① ≥0 ② <0

P_k −2 P_k +10

	P_{k+1}	X	y	set (y, x)
	–	32	8	(8, 32)
Case ①	4	33	7	(7, 33)
Case ②	2	34	6	(6, 34)
"	0	35	5	(5, 35)
Case 2	−2	36	5	(5, 36)
	8	37	4	(4, 37)
	6	38	3	(3, 38)

CamScanner

Ex.4 $(x_1, y_1) = (38, 3)$ & $(x_2, y_2) = (32, 8)$

① $|\Delta x| = |32 - 38| = |-6|$, $|\Delta y| = 8 - 3 = 5$

$\therefore \Delta x > \Delta y$ no swap

$\therefore \Delta x = -6$ x is dec. , y inc.

$P_0 = 2\Delta y - \Delta x = 10 - 6 = 4$

≥ 0 P_{k+1} < 0

$P_k - 2$ $P_k + 10$

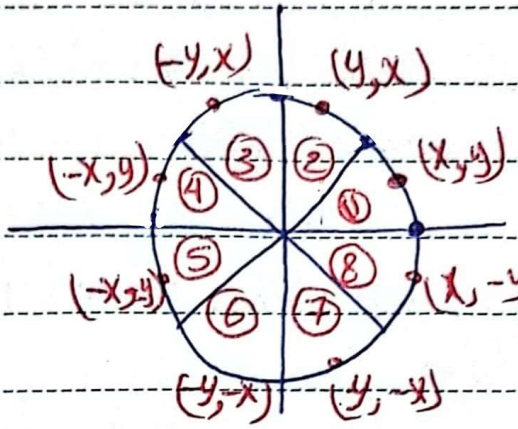
P_{k+1}	x	y	set (x, y)
-	38	3	(38, 3)
4	37	4	(37, 4)
2	36	5	(36, 5)
0	35	6	(35, 6)
-2	34	6	(34, 6)
8	33	7	(33, 7)
6	32	8	(32, 8)

CamScanner

Secs: Circle drawing

* Midpoint algorithm

Caculate points in one octant then drive the rest for the other 7 octants



Step 1: initial point $(0, r)$ (r ↓ radius)

Step 2: $P_0 = 1 - r$

Step 3:

if $P_k < 0$	$y_{k+1} = y_k$ (inc x) $P_{k+1} = P_k + 2x_{k+1} + 1$
$P_k > 0$	$y_{k+1} = y_k - 1$ (inc x) $P_{k+1} = P_k + 2x_{k+1} + 1 - 2y_{k+1}$

Step 4: repeat until $x \geq y$

EX: using Brensenham Circle with center (0,0) & $r = 10$, second octant

Sol

(1) initial point $(0, 10)$

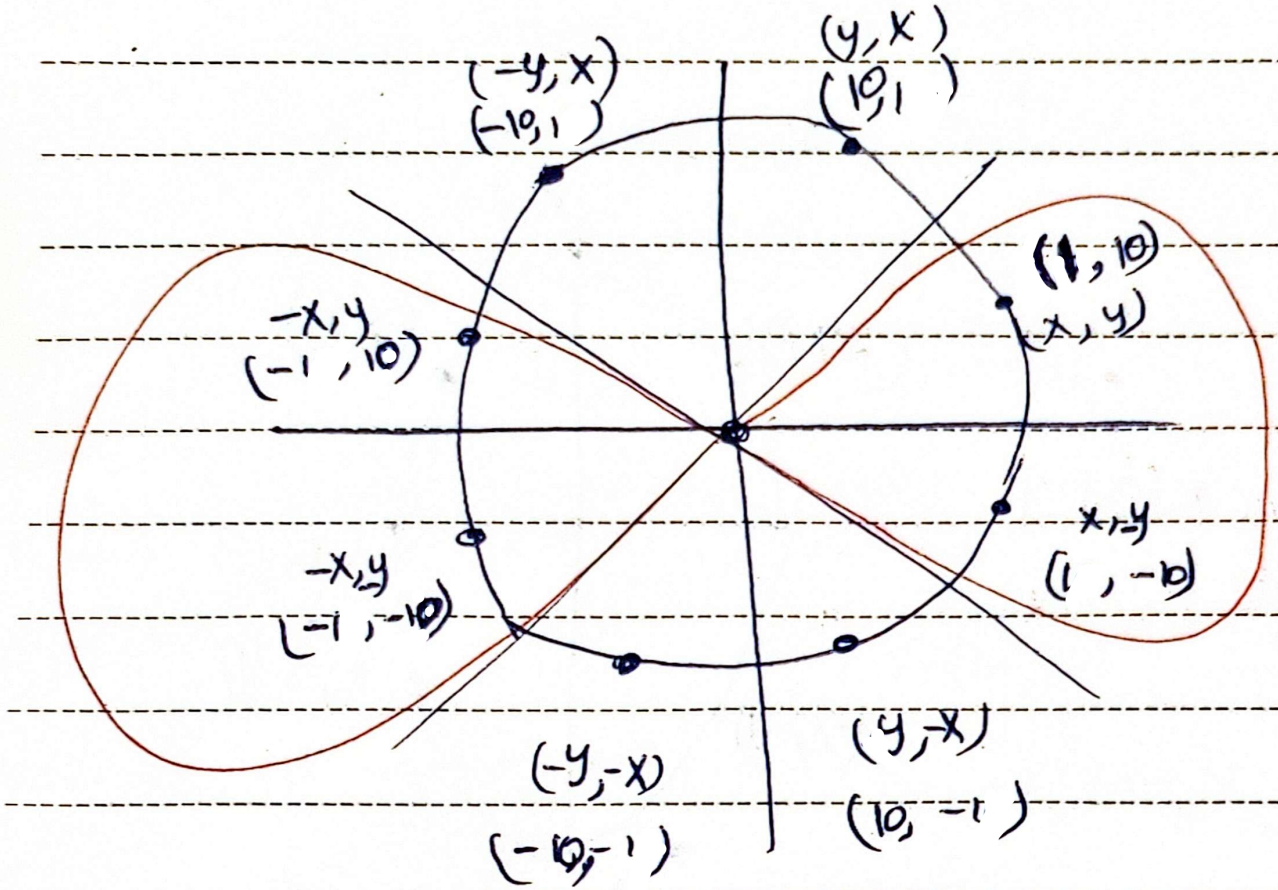
$P_0 = 1 - 10 = -9$

P_k ? < 0 → $y = y$, $P_{k+1} = P_k + 2x + 1$

> 0 → $y = y - 1$, $P_{k+1} = P_k + 2x + 1 - 2y$

k	P_k	x_{k+1}, y_{k+1}	$2x_{k+1}$	$2y_{k+1}$
0	-9	(1, 10)	2	20
1	$-9+3 = -6$	(2, 10)	4	20
2	-1	(3, 10)	6	20
	$-1+7 = 6$	(4, 9)	8	18
	-3	(5, 9)	10	18
	$-3+11 = 8$	(6, 8)	12	16
	5	(7, 7)	—	—

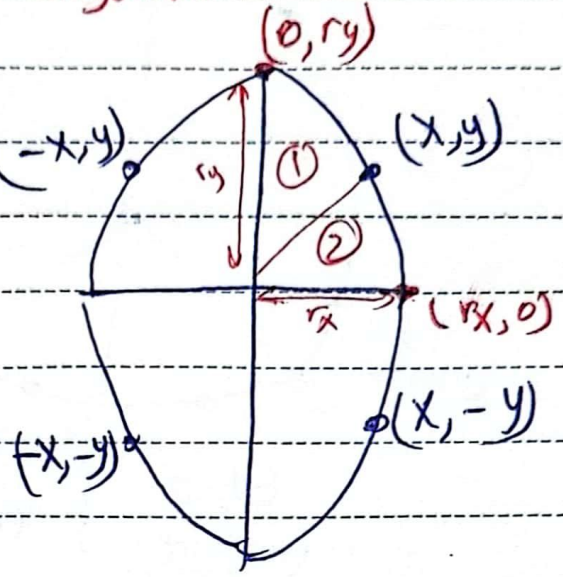
$x \geq y$



if the center wasn't $(0,0)$

then $(x_c + x, y_c + y)$

[2] Ellipse generation algorithm



Calculate

Region 1 (inc x), (Cal. y)

Region 2 (dec y), (Cal x)

Ex 1 Given input ellipse Prometer $r_x=8, r_y=6$

Sol

for region 1 ① $(x_0,y_0)=(0,6)$

② $P1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2 = -332$

P_k

>0 ① ② <0

$x = x+1$

$y = y-1$

$P_{k+1} = P_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$

$x = x+1$

$y = y$

$P_{k+1} = P_k + 2r_y^2 x_{k+1} + r_y^2$

$2 \times 6^2 \times 1$

repeat until $2r_y^2 x \geq 2r_x^2 y$ Case 2

k	$P1_k$	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	-332	(1, 6)	72	768
1	-224	(2,6)	144	768
2	-44	(3,6)	216	768
3	208	(4,5)	288	640
4	-108	(5,5)	360	640
5	288	(6,4)	432	512
6	244	(7,3)	504 >	384

STOP

R.I. من النقطة التي انتهت

for Region 2 $(x_0, y_0) = (7, 3)$

$$P_0^2 = r_y^2 (x_0 + \frac{1}{2})^2 + r_x^2 (y_0 - 1)^2 - r_x^2 * r_y^2$$

P_k

>0 ← → ≤ 0

$x = x$
$y = y-1$

$$P_{K+1} = P_k - 2r_x^2 y_{k+1} + r_x^2$$

$x = x+1$
$y = y-1$

$$P_{K+1} = P_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$$

repeat until we reach point → $y = 0$

K	P_k^2	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	-23	(8,2)	576	256
1	361	(8,1)	576	128
2	297	(8,0)	-	-

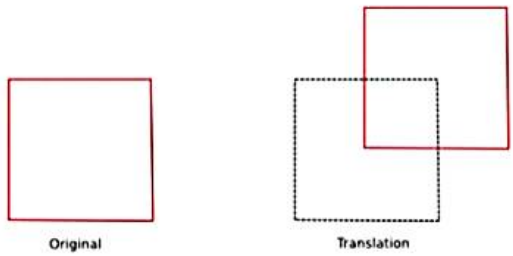
Stop

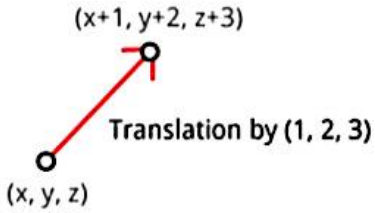
DAWENHA designs

CamScanner

Translation

To see why we're working with 4-by-1 vectors and subsequently 4-by-4 transformation matrices, let's see how a translation matrix is formed. A translation moves a vector a certain distance in a certain direction. glTranslated(x,y,z);



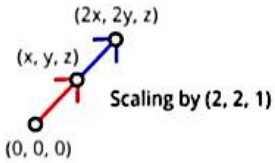


Without the fourth column and the bottom 1 value a translation wouldn't have been possible.

$$\begin{bmatrix} 1 & 0 & 0 & X \\ 0 & 1 & 0 & Y \\ 0 & 0 & 1 & Z \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x + X \cdot 1 \\ y + Y \cdot 1 \\ z + Z \cdot 1 \\ 1 \end{pmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- **Scaling**
- A scale transformation scales each of a vector's components by a (different) scalar. It is commonly used to shrink or stretch a vector as demonstrated below.





Scaling

- If you understand how the previous matrix was formed, it should not be difficult to come up with a matrix that scales a given vector by (SX,SY,SZ). If scaling factor > 1, then the object size is increased. If scaling factor < 1, then the object size is reduced. glScaled(x,y,z);

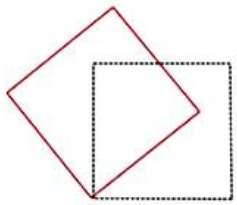
$$\begin{bmatrix} SX & 0 & 0 & 0 \\ 0 & SY & 0 & 0 \\ 0 & 0 & SZ & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} SX \cdot x \\ SY \cdot y \\ SZ \cdot z \\ 1 \end{pmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- If you think about it for a moment, you can see that scaling would also be possible with a mere 3-by-3 matrix.

CamScanner

- **Rotation**
- A rotation transformation rotates a vector around the origin (0,0,0) using a given axis and angle.
- Objects can be rotated around any given axis, but for now only the X, Y and Z axis are important.
  - glRotated(angel, x, y, z) //to rotate around z axes –> (0,0,1)



Rotation

- The matrices for rotating around the three axes are specified here. The rotation angle is indicated by the theta (ϑ).
- Rotation around X-axis:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ \cos\theta \cdot y - \sin\theta \cdot z \\ \sin\theta \cdot y + \cos\theta \cdot z \\ 1 \end{pmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Rotation around Y-axis:

$$\begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta \cdot x + \sin\theta \cdot z \\ y \\ -\sin\theta \cdot x + \cos\theta \cdot z \\ 1 \end{pmatrix}$$

- Rotation around Z-axis:

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta \cdot x - \sin\theta \cdot y \\ \sin\theta \cdot x + \cos\theta \cdot y \\ z \\ 1 \end{pmatrix}$$

shearing:

CamScanner

$$\text{shear in x: } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\text{shear in y: } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Transformations exercises:

A triangle is defined by three points A(2,2), B(3,5), c(4,4) what's the transformation coordinate after horizontal reflection.

A) A(-2,2), B(-3,5), c(-4,4)

B) A(2,-2), B(3,-5), c(4,-4)

C) A(-2,-2), B(-3,-5), c(-4,-4)

ANSWER:B

15) Rotate Point p by angle 50 degree, then rotate it by angle -60 then scale it to half vertically

P'=s(1,0.5).r(50+-60)→P'=s(1,0.5).r(-10)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} \cos -10 & -\sin -10 & 0 \\ \sin -10 & \cos -10 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

16) Rotate point p by angle 20 around point (5,6) then rotate angle by -50 around point (3,7)

CamScanner

P'= T(3,7). R(-50). T(-3,-7). T(5,6). R(20). T(-5,-6). P

P'= T(3,7). R(-50). T(2,-1). R(20). T(-5,-6). P

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 7 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} \cos(-50) & -\sin(-50) & 0 \\ \sin(-50) & \cos(-50) & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 0 & 0 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} \cos 20 & -\sin 20 & 0 \\ \sin 20 & \cos 20 & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 0 & 0 & -5 \\ 0 & 0 & -6 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

17) Magnify a triangle with vertices A(1,1), B(2, 3) and c(7, 3) to twice its size while keeping c(7,3) fixed

P_A'= T(7,3).S(2,2).T(-7,-3). A→ $\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 1 & 0 & -7 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$

P_B'= (7,3).S(2,2).T(-7,-3). B → $\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 1 & 0 & -7 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$

P_C'= (7,3).S(2,2).T(-7,-3). C → $\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 1 & 0 & -7 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 7 \\ 3 \\ 1 \end{bmatrix}$

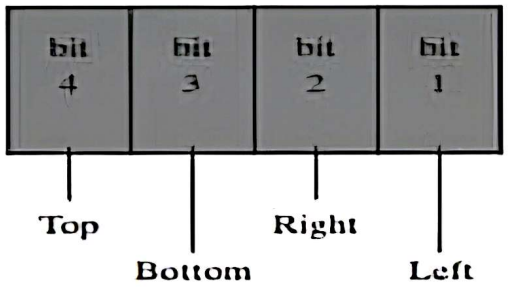
Example in 3D!

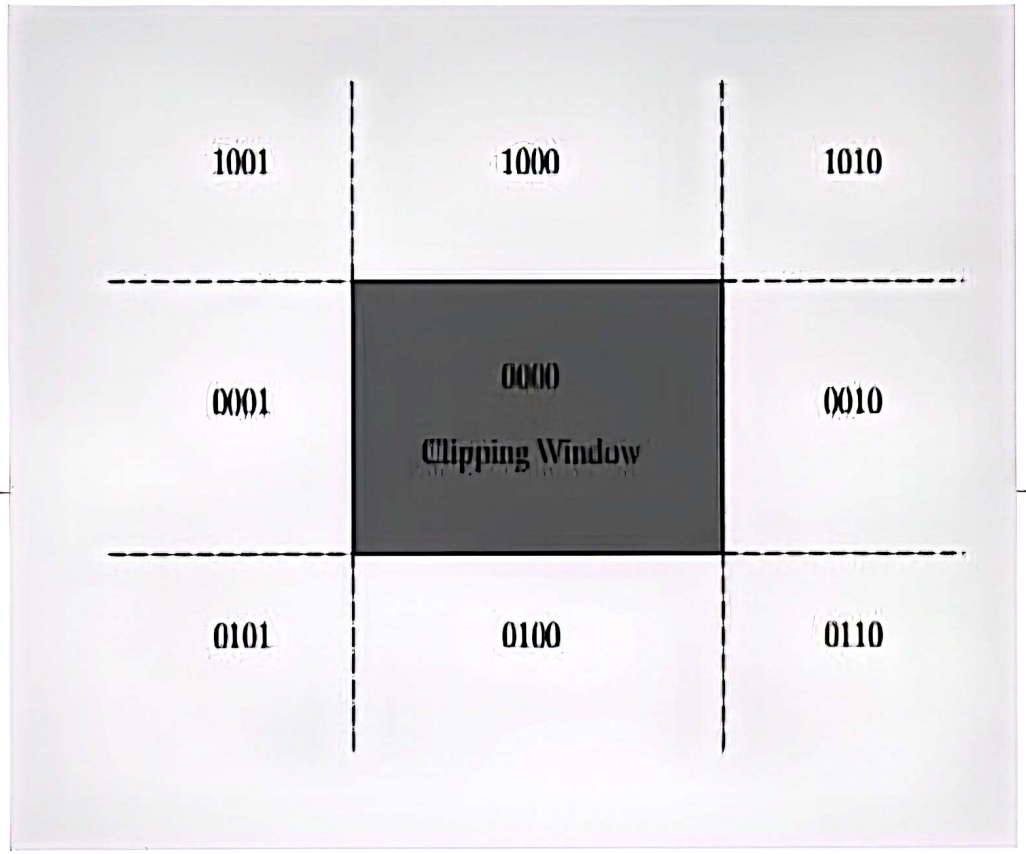
- Let's take some 3D object, say a cube centered at (2,2,2)
- Rotate clockwise in object's space by 30° around *x* axis, 60° around *y*, and 90° around *z*
- Scale in object space by 1 in the *x*, 2 in the *y*, and 3 in the *z*
- Translate by (2,2,4) in world space
- Transformation sequence: $TT_0^{-1}S_{xy}R_{xy}R_{zx}R_{yz}T_o$, where T_0 translates to (0,0):

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 4 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{T} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{T_0^{-1}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{S_{xy}} \underbrace{\begin{bmatrix} \cos 90 & \sin 90 & 0 & 0 \\ -\sin 90 & \cos 90 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{R_{xy}} \underbrace{\begin{bmatrix} \cos 60 & 0 & -\sin 60 & 0 \\ 0 & 1 & 0 & 0 \\ \sin 60 & 0 & \cos 60 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{R_{zx}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos 30 & \sin 30 & 0 \\ 0 & -\sin 30 & \cos 30 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{R_{yz}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & -2 \\ 0 & 1 & 0 & -2 \\ 0 & 0 & 1 & -2 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{T_0}$$

CamScanner

1-COHEN-SUTHERLAND LINE CLIPING





CamScanner

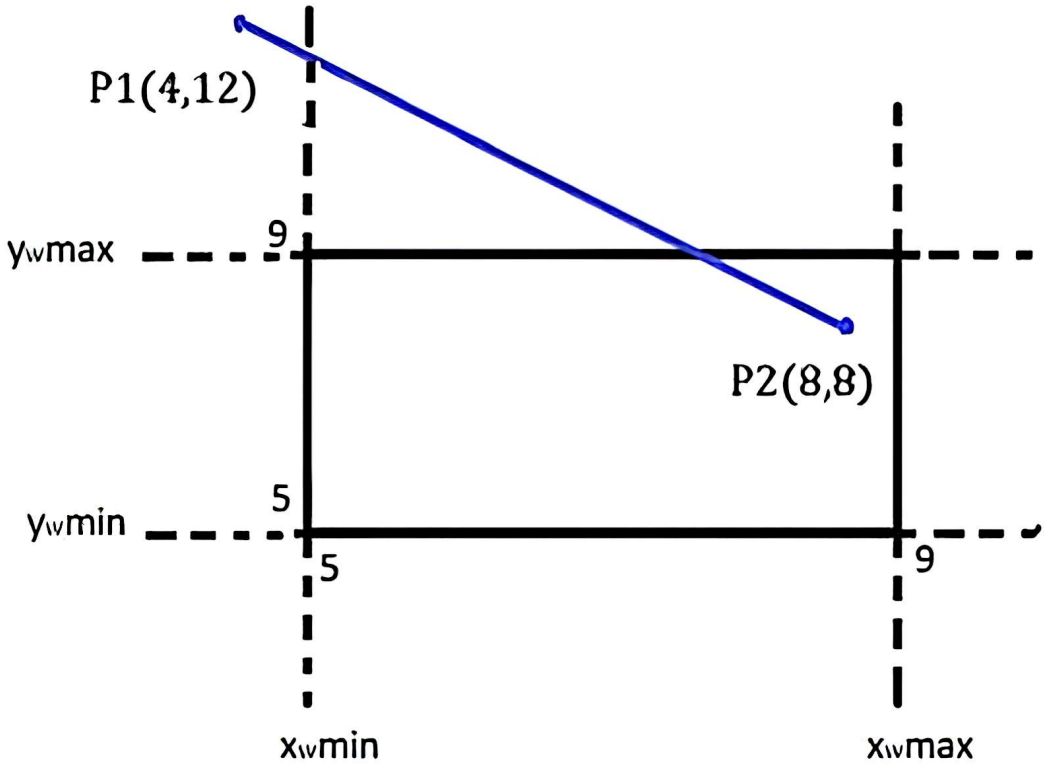
1. Determine region code for endpoints
2. See OR gate bet the two region codes if it got 0000 then the line is completely inside
3. See AND gate bet the two region codes if at least one bit is 1 then the line is completely outside (eliminated)
4. Else there is intersection , calculate the new points after clipping repeat until this chick is true for with the outside pointes
   1. Determine slope $m = \frac{y_2 - y_1}{x_2 - x_1}$
   2. the intersection point with a vertical clipping border line
      $ynew = y1 + m(x - x1)$ ..*x* value set to either *xw*min or *xw*max
   3. the intersection with a horizontal border, the *x* coordinate can be calculated as
      $xnew = x1 + \frac{y - y_1}{m}$ ..*y value* set either to *yw*min or to *yw*max.

$$xw_{min} \le x \le xw_{max}$$
$$yw_{min} \le y \le yw_{max}$$

CamScanner

Consider the window size from 5 to 9. clip the following line

i.(4,12) (8,8)



CamScanner

Line P1(4,12) to P2(8,8)

Region code for P1 1001(outside)

Region code for P2 0000 (inside)

So line is inside the window

P1 is outside so calculate new value for P1

$x_1 = 4\ y_1 = 12\ x_2 = 8\ y_2 = 8$

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{8 - 12}{8 - 4} = -1$$

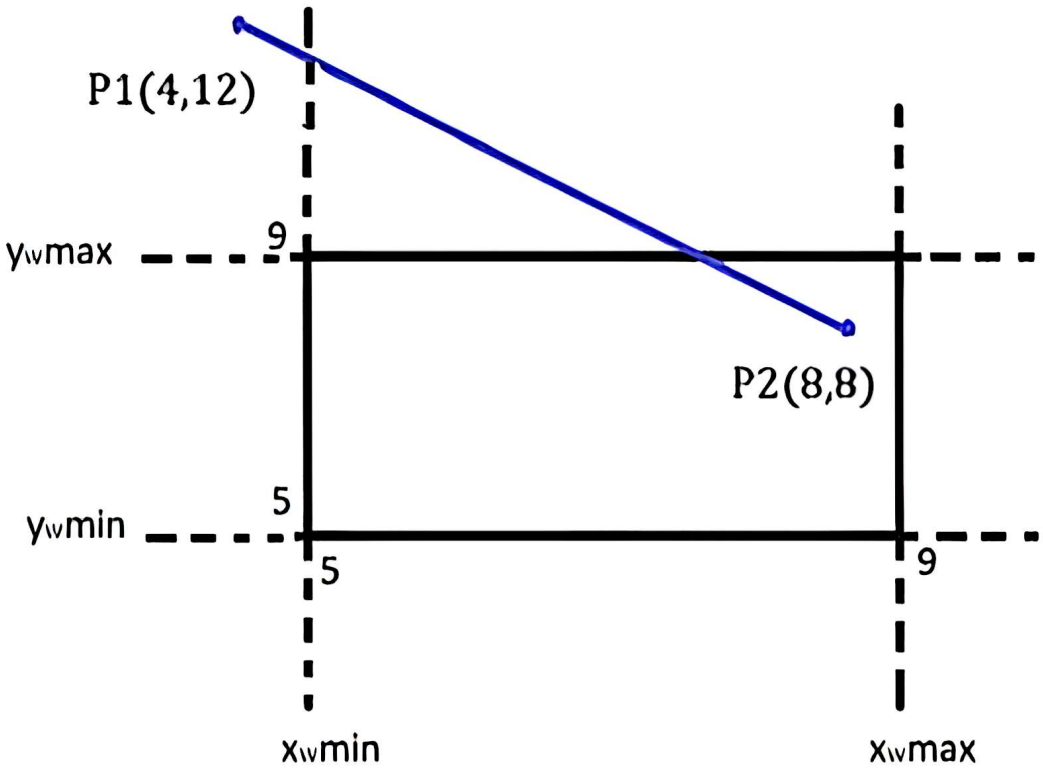
Line intersect with $x_w min$

Find y

$y = y_1 + m(x - x_1)$

Here x=$x_w min$=5

$y = y_1 + m(x - x_1)$=12+(-1)(5-4)=12-1=11



CamScanner

$y = 11$

So the new point is (5,11)

$x_w min \leq x \leq x_w max$
$y_w min \leq y \leq y_w max$

$x_w min \leq 5 \leq x_w max$
$y_w min \leq 11 \leq y_w max$

Condition Fails Apply algorithm once again

So the new line is from (5,11) to (8,8)

$x_1 = 5\ y_1 = 11\ x_2 = 8\ y_2 = 8$

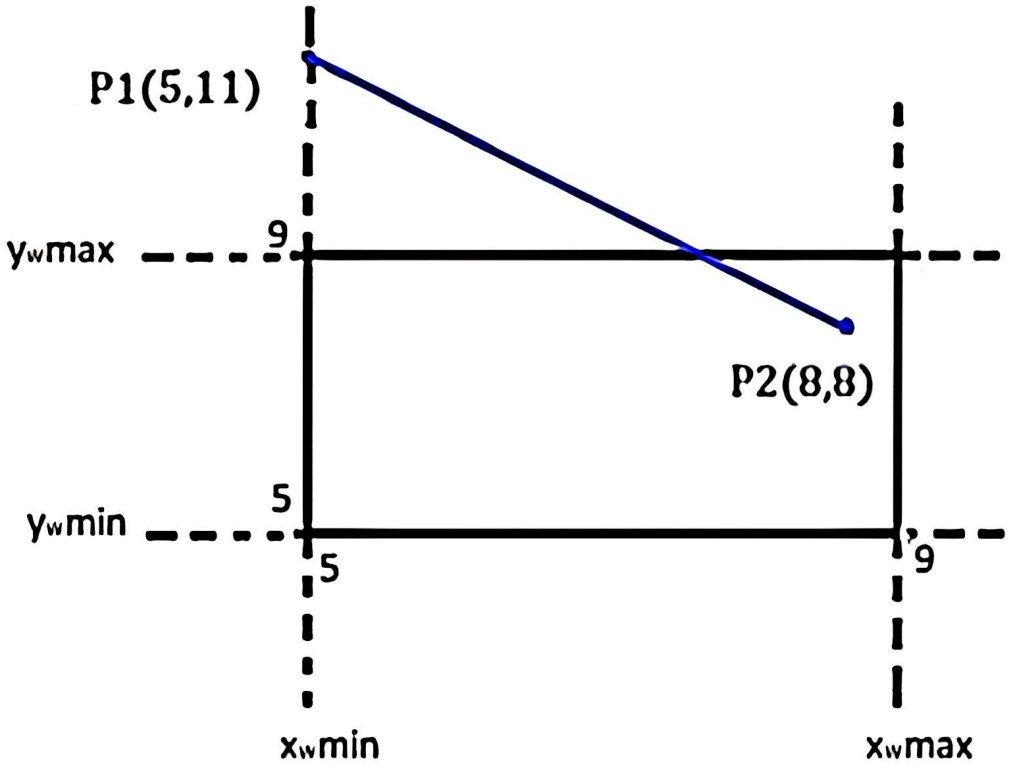
$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{8 - 11}{8 - 1} = -1$$

Now the line is intersecting with $y_w max$

Find x

$$x = x_1 + \frac{(y - y_1)}{m}$$

Here $y = y_w max = 9$



CamScanner